

Import Libraries

```
In [1]: # Library pandas importing
import pandas as pd
```

Read the Dataset

```
In [2]: # Load Movies, Tags, Ratings data set
movies_data = pd.read_csv('Datasets/movie.csv')
tags_data = pd.read_csv('Datasets/tag.csv')
ratings_data = pd.read_csv('Datasets/rating.csv')
```

```
In [3]: movies_data.head()
```

```
Out[3]:
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

```
In [4]: movies_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27278 entries, 0 to 27277
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   movieId     27278 non-null  int64
1   title       27278 non-null  object
2   genres      27278 non-null  object
dtypes: int64(1), object(2)
memory usage: 639.5+ KB
```

```
In [5]: movies_data.describe()
```

Out [5]:

movielid	
count	27278.000000
mean	59855.480570
std	44429.314697
min	1.000000
25%	6931.250000
50%	68068.000000
75%	100293.250000
max	131262.000000

In [6]: `movies_data.shape`

Out[6]: (27278, 3)

In [7]: `tags_data.shape`

Out[7]: (465564, 4)

In [8]: `tags_data.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 465564 entries, 0 to 465563
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   userId      465564 non-null  int64
1   movieId     465564 non-null  int64
2   tag         465548 non-null  object
3   timestamp   465564 non-null  object
dtypes: int64(2), object(2)
memory usage: 14.2+ MB
```

In [9]: `tags_data.head()`

Out[9]:

	userId	movielid	tag	timestamp
0	18	4141	Mark Waters	2009-04-24 18:19:40
1	65	208	dark hero	2013-05-10 01:41:18
2	65	353	dark hero	2013-05-10 01:41:19
3	65	521	noir thriller	2013-05-10 01:39:43
4	65	592	dark hero	2013-05-10 01:41:18

In [10]: `ratings_data.shape`

```
Out[10]: (20000263, 4)
```

```
In [11]: ratings_data.head()
```

```
Out[11]:
```

	userId	movieId	rating	timestamp
0	1	2	3.5	2005-04-02 23:53:47
1	1	29	3.5	2005-04-02 23:31:16
2	1	32	3.5	2005-04-02 23:33:39
3	1	47	3.5	2005-04-02 23:32:07
4	1	50	3.5	2005-04-02 23:29:40

```
In [12]: ratings_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20000263 entries, 0 to 20000262
Data columns (total 4 columns):
#   Column      Dtype
---  -
0   userId      int64
1   movieId     int64
2   rating      float64
3   timestamp   object
dtypes: float64(1), int64(2), object(1)
memory usage: 610.4+ MB
```

```
In [13]: # Remove timestamp column for this data analysis
```

```
print(movies_data.columns)
print(tags_data.columns)
print(ratings_data.columns)
```

```
Index(['movieId', 'title', 'genres'], dtype='object')
Index(['userId', 'movieId', 'tag', 'timestamp'], dtype='object')
Index(['userId', 'movieId', 'rating', 'timestamp'], dtype='object')
```

```
In [14]: del tags_data['timestamp']
del ratings_data['timestamp']
```

```
In [15]: print(movies_data.columns)
print(tags_data.columns)
print(ratings_data.columns)
```

```
Index(['movieId', 'title', 'genres'], dtype='object')
Index(['userId', 'movieId', 'tag'], dtype='object')
Index(['userId', 'movieId', 'rating'], dtype='object')
```

Data Structures

```
In [16]: # Series
row_0 = tags_data.iloc[0]
row_0
```

```
Out[16]:  userId          18
         movieId       4141
         tag      Mark Waters
         Name: 0, dtype: object
```

```
In [17]: type(row_0)
```

```
Out[17]: pandas.core.series.Series
```

```
In [18]: row_0.index
```

```
Out[18]: Index(['userId', 'movieId', 'tag'], dtype='object')
```

```
In [19]: 'rating' in row_0
```

```
Out[19]: False
```

```
In [20]: 'userId' in row_0
```

```
Out[20]: True
```

```
In [21]: row_0.name
```

```
Out[21]: 0
```

```
In [22]: row_0 = row_0.rename('first_name')
```

```
In [23]: row_0.name
```

```
Out[23]: 'first_name'
```

```
In [24]: tags_data.head()
```

```
Out[24]:
```

	userId	movieId	tag
0	18	4141	Mark Waters
1	65	208	dark hero
2	65	353	dark hero
3	65	521	noir thriller
4	65	592	dark hero

```
In [25]: tags_data.index
```

```
Out[25]: RangeIndex(start=0, stop=465564, step=1)
```

```
In [26]: tags_data.columns
```

```
Out[26]: Index(['userId', 'movieId', 'tag'], dtype='object')
```

```
In [27]: tags_data.iloc[[0,11,500]]
```

```
Out[27]:
```

	userId	movieId	tag
0	18	4141	Mark Waters
11	65	1783	noir thriller
500	342	55908	entirely dialogue

Descriptive Statistics

```
In [28]: ratings_data['rating'].describe()
```

```
Out[28]:
```

count	2.000026e+07
mean	3.525529e+00
std	1.051989e+00
min	5.000000e-01
25%	3.000000e+00
50%	3.500000e+00
75%	4.000000e+00
max	5.000000e+00

Name: rating, dtype: float64

```
In [29]: ratings_data.describe()
```

```
Out[29]:
```

	userId	movieId	rating
count	2.000026e+07	2.000026e+07	2.000026e+07
mean	6.904587e+04	9.041567e+03	3.525529e+00
std	4.003863e+04	1.978948e+04	1.051989e+00
min	1.000000e+00	1.000000e+00	5.000000e-01
25%	3.439500e+04	9.020000e+02	3.000000e+00
50%	6.914100e+04	2.167000e+03	3.500000e+00
75%	1.036370e+05	4.770000e+03	4.000000e+00
max	1.384930e+05	1.312620e+05	5.000000e+00

```
In [30]: ratings_data['rating'].mean()
```

```
Out[30]: np.float64(3.5255285642993797)
```

```
In [31]: ratings_data['rating'].min()
```

```
Out[31]: 0.5
```

```
In [32]: ratings_data['rating'].max()
```

Out[32]: 5.0

```
In [33]: ratings_data['rating'].mode()
```

Out[33]: 0 4.0
Name: rating, dtype: float64

```
In [34]: ratings_data.corr()
```

Out[34]:

	userId	movieId	rating
userId	1.000000	-0.000850	0.001175
movieId	-0.000850	1.000000	0.002606
rating	0.001175	0.002606	1.000000

```
In [36]: filter_data1 = ratings_data['rating'] > 10  
print(filter_data1)
```

```
0      False  
1      False  
2      False  
3      False  
4      False  
...  
20000258 False  
20000259 False  
20000260 False  
20000261 False  
20000262 False  
Name: rating, Length: 20000263, dtype: bool
```

```
In [38]: print(filter_data1.any())
```

False

```
In [39]: filter_data2 = ratings_data['rating'] > 0  
print(filter_data2.all())
```

True

Data Cleaning: Handling Missing Data

```
In [40]: movies_data.shape
```

Out[40]: (27278, 3)

```
In [41]: movies_data.isnull().any().any()
```

Out[41]: np.False_

```
In [42]: ratings_data.shape
```

Out[42]: (20000263, 3)

```
In [43]: ratings_data.isnull().any().any()
```

Out[43]: np.False_

```
In [44]: tags_data.shape
```

Out[44]: (465564, 3)

```
In [45]: tags_data.isnull().any().any()
```

Out[45]: np.True_

```
In [46]: tags_data = tags_data.dropna()
```

```
In [47]: tags_data.isnull().any().any()
```

Out[47]: np.False_

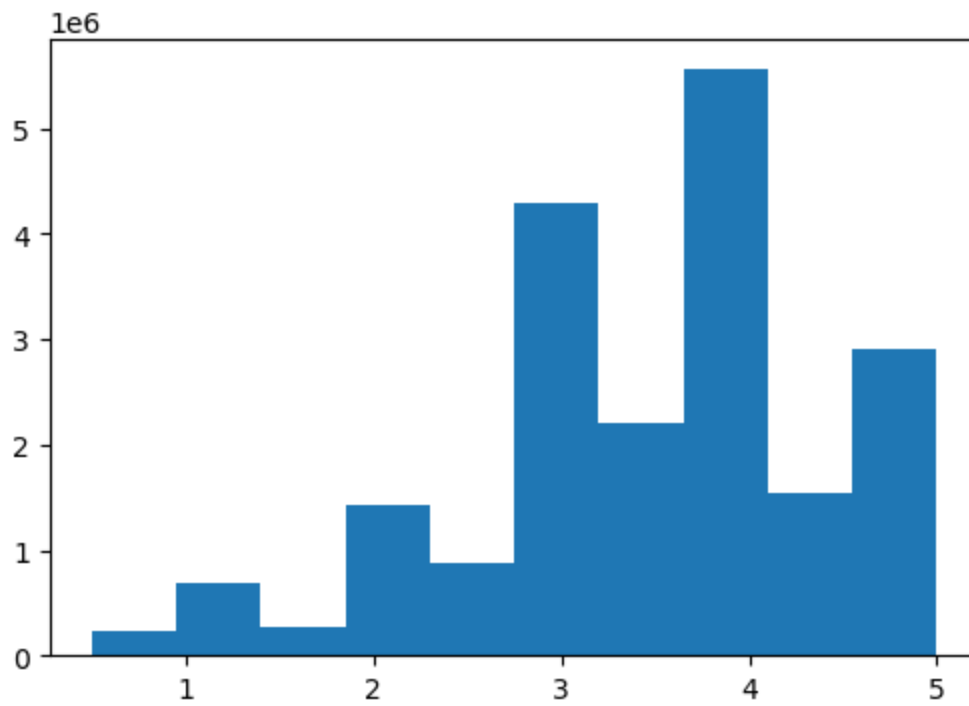
```
In [48]: tags_data.shape
```

Out[48]: (465548, 3)

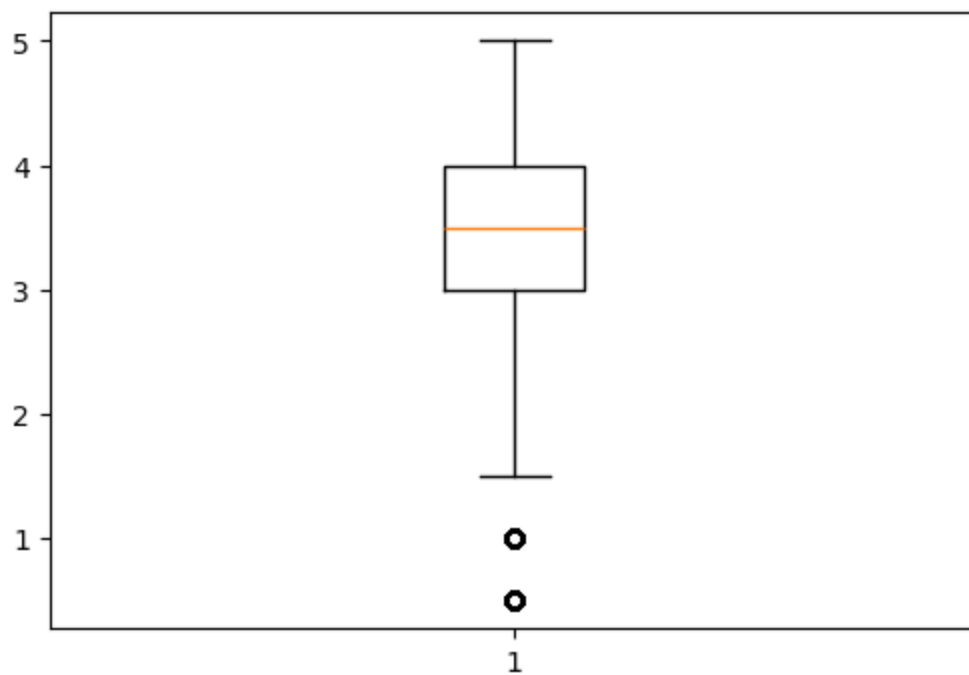
Data Visualization

```
In [49]: import matplotlib.pyplot as plt
%matplotlib inline
plt.rcParams['figure.figsize'] = (6,4)
```

```
In [50]: plt.hist(ratings_data['rating'])
plt.show()
```



```
In [51]: plt.boxplot(ratings_data['rating'])  
plt.show()
```



Slicing Out Columns

```
In [52]: tags_data['tag'].head()
```



```
Out[52]: 0      Mark Waters
1      dark hero
2      dark hero
3      noir thriller
4      dark hero
Name: tag, dtype: object
```

```
In [53]: movies_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27278 entries, 0 to 27277
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   movieId     27278 non-null    int64
1   title       27278 non-null    object
2   genres      27278 non-null    object
dtypes: int64(1), object(2)
memory usage: 639.5+ KB
```

```
In [54]: movies_data[['title', 'genres']].head()
```

```
Out[54]:
```

	title	genres
0	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	Jumanji (1995)	Adventure Children Fantasy
2	Grumpier Old Men (1995)	Comedy Romance
3	Waiting to Exhale (1995)	Comedy Drama Romance
4	Father of the Bride Part II (1995)	Comedy

```
In [55]: ratings_data[-10:]
```

```
Out[55]:
```

	userId	movieId	rating
20000253	138493	60816	4.5
20000254	138493	61160	4.0
20000255	138493	65682	4.5
20000256	138493	66762	4.5
20000257	138493	68319	4.5
20000258	138493	68954	4.5
20000259	138493	69526	4.5
20000260	138493	69644	3.0
20000261	138493	70286	5.0
20000262	138493	71619	2.5

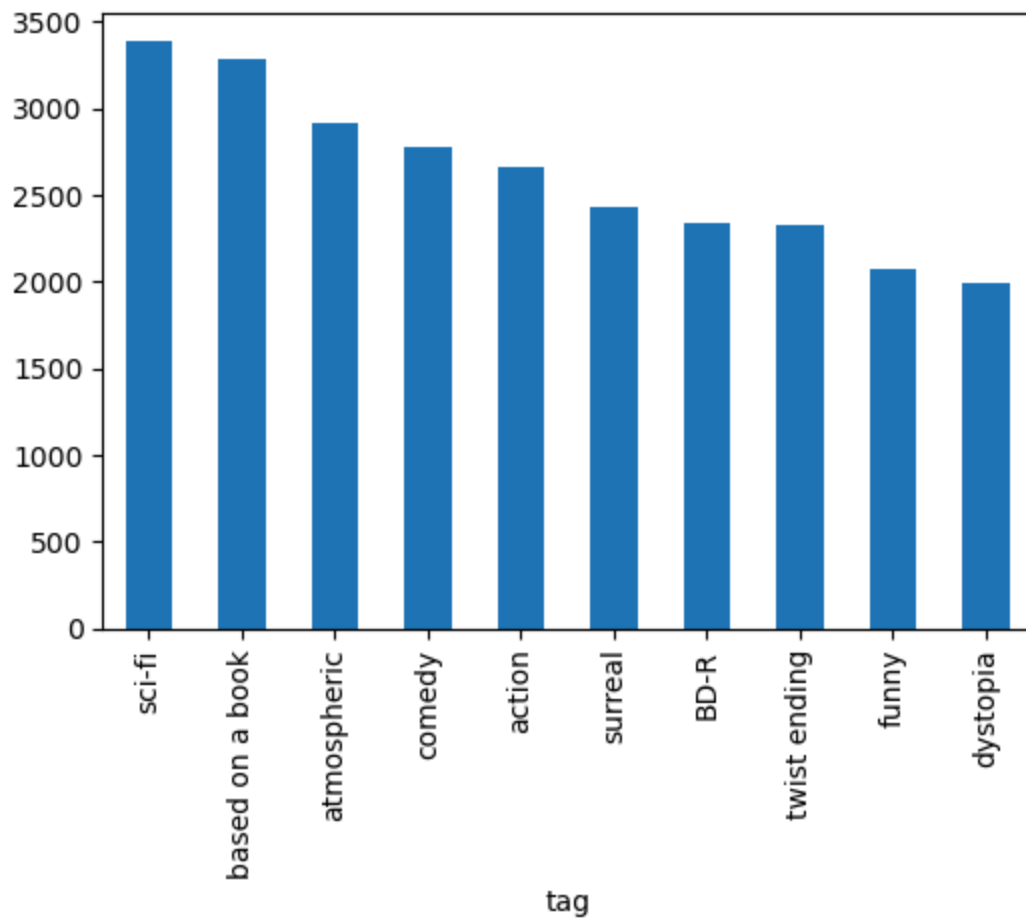
```
In [58]: tags_count = tags_data['tag'].value_counts()
tags_count[-10:]
```

```
Out[58]: tag
missing child          1
Ron Moore              1
Citizen Kane          1
mullet                1
biker gang            1
Paul Adelstein        1
the wig               1
killer fish           1
genetically modified monsters 1
topless scene         1
Name: count, dtype: int64
```

```
In [66]: tags_count
```

```
Out[66]: tag
sci-fi                3384
based on a book       3281
atmospheric           2917
comedy                2779
action                2657
...
Paul Adelstein        1
the wig               1
killer fish           1
genetically modified monsters 1
topless scene         1
Name: count, Length: 38643, dtype: int64
```

```
In [65]: tags_count[:10].plot(kind='bar')
plt.show()
```



Filters for Selecting Rows

```
In [71]: is_highly_rated = ratings_data [ ratings_data['rating'] >= 5 ]  
is_highly_rated[30:50]
```

Out [71]:

	userId	movieId	rating
239	3	50	5.0
242	3	175	5.0
244	3	223	5.0
245	3	260	5.0
246	3	316	5.0
247	3	318	5.0
248	3	329	5.0
252	3	457	5.0
253	3	480	5.0
254	3	490	5.0
256	3	541	5.0
258	3	593	5.0
263	3	858	5.0
264	3	904	5.0
267	3	924	5.0
268	3	953	5.0
271	3	1060	5.0
272	3	1073	5.0
275	3	1084	5.0
276	3	1089	5.0

```
In [78]: movies_geners = movies_data[movies_data['genres'].str.contains('Action')]
movies_geners[5:15]
```

Out [78]:

	movieid	title	genres
22	23	Assassins (1995)	Action Crime Thriller
41	42	Dead Presidents (1995)	Action Crime Drama
43	44	Mortal Kombat (1995)	Action Adventure Fantasy
50	51	Guardian Angel (1994)	Action Drama Thriller
65	66	Lawnmower Man 2: Beyond Cyberspace (1996)	Action Sci-Fi Thriller
69	70	From Dusk Till Dawn (1996)	Action Comedy Horror Thriller
70	71	Fair Game (1995)	Action
75	76	Screamers (1995)	Action Sci-Fi Thriller
77	78	Crossing Guard, The (1995)	Action Crime Drama Thriller
85	86	White Squall (1996)	Action Adventure Drama

In [80]: `movies_geners.head(15)`

Out [80]:

	movieid	title	genres
5	6	Heat (1995)	Action Crime Thriller
8	9	Sudden Death (1995)	Action
9	10	GoldenEye (1995)	Action Adventure Thriller
14	15	Cutthroat Island (1995)	Action Adventure Romance
19	20	Money Train (1995)	Action Comedy Crime Drama Thriller
22	23	Assassins (1995)	Action Crime Thriller
41	42	Dead Presidents (1995)	Action Crime Drama
43	44	Mortal Kombat (1995)	Action Adventure Fantasy
50	51	Guardian Angel (1994)	Action Drama Thriller
65	66	Lawnmower Man 2: Beyond Cyberspace (1996)	Action Sci-Fi Thriller
69	70	From Dusk Till Dawn (1996)	Action Comedy Horror Thriller
70	71	Fair Game (1995)	Action
75	76	Screamers (1995)	Action Sci-Fi Thriller
77	78	Crossing Guard, The (1995)	Action Crime Drama Thriller
85	86	White Squall (1996)	Action Adventure Drama

Group By and Aggregate

```
In [82]: ratings_data.columns
```

```
Out[82]: Index(['userId', 'movieId', 'rating'], dtype='object')
```

```
In [86]: ratings_groups = ratings_data[['movieId', 'rating']].groupby('rating').count()  
ratings_groups
```

```
Out[86]:
```

	movieId
rating	
0.5	239125
1.0	680732
1.5	279252
2.0	1430997
2.5	883398
3.0	4291193
3.5	2200156
4.0	5561926
4.5	1534824
5.0	2898660

```
In [88]: average_rating = ratings_data[['movieId', 'rating']].groupby('movieId').mean()  
average_rating
```

Out [88]:

	rating
movieId	
1	3.921240
2	3.211977
3	3.151040
4	2.861393
5	3.064592
...	...
131254	4.000000
131256	4.000000
131258	2.500000
131260	3.000000
131262	4.000000

26744 rows × 1 columns

In [89]: `ratings_data.head(20)`

Out [89]:

	userId	movieId	rating
0	1	2	3.5
1	1	29	3.5
2	1	32	3.5
3	1	47	3.5
4	1	50	3.5
5	1	112	3.5
6	1	151	4.0
7	1	223	4.0
8	1	253	4.0
9	1	260	4.0
10	1	293	4.0
11	1	296	4.0
12	1	318	4.0
13	1	337	3.5
14	1	367	3.5
15	1	541	4.0
16	1	589	3.5
17	1	593	3.5
18	1	653	3.0
19	1	919	3.5

```
In [91]: movie_count = ratings_data.groupby('movieId').count()  
movie_count.tail()
```

Out [91]:

	userId	rating
movieId		
131254	1	1
131256	1	1
131258	1	1
131260	1	1
131262	1	1

Merge Dataframes


```
In [92]: tags_data.head()
```

```
Out[92]:
```

	userId	movieId	tag
0	18	4141	Mark Waters
1	65	208	dark hero
2	65	353	dark hero
3	65	521	noir thriller
4	65	592	dark hero

```
In [93]: movies_data.head()
```

```
Out[93]:
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

```
In [96]: total_data = movies_data.merge(tags_data,on='movieId',how='inner')
total_data.head()
```

```
Out[96]:
```

	movieId	title	genres	userId	tag
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1644	Watched
1	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1741	computer animation
2	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1741	Disney animated feature
3	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1741	Pixar animation
4	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1741	TÃ©a Leoni does not star in this movie

Combine aggregation, merging, and filters to get useful analytics

```
In [101... average_rating = ratings_data.groupby('movieId',as_index=False).mean()
del average_rating['userId']
average_rating.head()
```

```
Out[101...   movieId  rating
0         1  3.921240
1         2  3.211977
2         3  3.151040
3         4  2.861393
4         5  3.064592
```

```
In [103... box_office = movies_data.merge(average_rating, on = "movieId",how="inner")
box_office.tail()
```

```
Out[103...   movieId  title  genres  rating
26739  131254  Kein Bund für's Leben (2007)  Comedy  4.0
26740  131256  Feuer, Eis & Dosenbier (2002)  Comedy  4.0
26741  131258  The Pirates (2014)  Adventure  2.5
26742  131260  Rentun Ruusu (2001)  (no genres listed)  3.0
26743  131262  Innocence (2014)  Adventure|Fantasy|Horror  4.0
```

```
In [112... high_rated = box_office[box_office['rating'] >= 4.0]
high_rated.head()
```

```
Out[112...   movieId  title  genres  rating
27         28  Persuasion (1995)  Drama|Romance  4.057546
46         47  Seven (a.k.a. Se7en) (1995)  Mystery|Thriller  4.053493
49         50  Usual Suspects, The (1995)  Crime|Mystery|Thriller  4.334372
81         82  Antonia's Line (Antonia) (1995)  Comedy|Drama  4.004925
108        110  Braveheart (1995)  Action|Drama|War  4.042534
```

```
In [113... is_Adventure = box_office[box_office['genres'].str.contains('Adventure')]
is_Adventure.head()
```

Out [113...

	movieid	title	genres	rating
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	3.921240
1	2	Jumanji (1995)	Adventure Children Fantasy	3.211977
7	8	Tom and Huck (1995)	Adventure Children	3.142049
9	10	GoldenEye (1995)	Action Adventure Thriller	3.430029
12	13	Balto (1995)	Adventure Animation Children	3.272416

In [133...

```
box_office[(box_office['genres'].str.contains('Adventure')) & (box_office['r
```

Out [133...

	movieid	title	genres	rating
257	260	Star Wars: Episode IV - A New Hope (1977)	Action Adventure Sci-Fi	4.190672
593	599	Wild Bunch, The (1969)	Adventure Western	4.004726
708	720	Wallace & Gromit: The Best of Aardman Animatio...	Adventure Animation Comedy	4.109473
891	908	North by Northwest (1959)	Action Adventure Mystery Romance Thriller	4.233538
952	969	African Queen, The (1951)	Adventure Comedy Romance War	4.101558

In [128...

```
is_highly_rated = box_office['rating'] >= 4.0
is_Adventure = box_office['genres'].str.contains('Adventure')
box_office[is_Adventure & is_highly_rated][-5:]
```

Out [128...	movieid	title	genres	rating
26611	130586	Itinerary of a Spoiled Child (1988)	Adventure Drama	4.5
26655	130996	The Beautiful Story (1992)	Adventure Drama Fantasy	5.0
26667	131050	Stargate SG-1 Children of the Gods - Final Cut...	Adventure Sci-Fi Thriller	5.0
26736	131248	Brother Bear 2 (2006)	Adventure Animation Children Comedy Fantasy	4.0
26743	131262	Innocence (2014)	Adventure Fantasy Horror	4.0

Vectorized String Operations

In [134... `movies_data.head()`

Out [134...	movieid	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

Split 'genres' into multiple columns

In [135... `movie_genres = movies_data['genres'].str.split('|', expand=True)`

In [136... `movie_genres`

Out [136...

	0	1	2	3	4	5	6	7	8	
0	Adventure	Animation	Children	Comedy	Fantasy	None	None	None	None	No
1	Adventure	Children	Fantasy	None	None	None	None	None	None	No
2	Comedy	Romance	None	None	None	None	None	None	None	No
3	Comedy	Drama	Romance	None	None	None	None	None	None	No
4	Comedy	None	None	None	None	None	None	None	None	No
...	...	...	...	...	...	...	...	...	...	
27273	Comedy	None	None	None	None	None	None	None	None	No
27274	Comedy	None	None	None	None	None	None	None	None	No
27275	Adventure	None	None	None	None	None	None	None	None	No
27276	(no genres listed)	None	None	None	None	None	None	None	None	No
27277	Adventure	Fantasy	Horror	None	None	None	None	None	None	No

27278 rows x 10 columns

Add a new column for comedy genre flag

In [137...

```
movie_genres['isComedy'] = movies_data['genres'].str.contains('Comedy')
movie_genres[:10]
```

Out [137...

	0	1	2	3	4	5	6	7	8	9	i
0	Adventure	Animation	Children	Comedy	Fantasy	None	None	None	None	None	
1	Adventure	Children	Fantasy	None	None	None	None	None	None	None	
2	Comedy	Romance	None	None	None	None	None	None	None	None	
3	Comedy	Drama	Romance	None	None	None	None	None	None	None	
4	Comedy	None	None	None	None	None	None	None	None	None	
5	Action	Crime	Thriller	None	None	None	None	None	None	None	
6	Comedy	Romance	None	None	None	None	None	None	None	None	
7	Adventure	Children	None	None	None	None	None	None	None	None	
8	Action	None	None	None	None	None	None	None	None	None	
9	Action	Adventure	Thriller	None	None	None	None	None	None	None	

Extract year from title e.g. (2007)

```
In [139... movies_data.head()
```

```
Out[139...      movied      title      genres
```

0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

```
In [162... movies_data['year'] = movies_data['title'].str.extract(r"\((.*)\)")
movies_data.head()
```

```
Out[162...      movied      title      genres  year
```

0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1995
1	2	Jumanji (1995)	Adventure Children Fantasy	1995
2	3	Grumpier Old Men (1995)	Comedy Romance	1995
3	4	Waiting to Exhale (1995)	Comedy Drama Romance	1995
4	5	Father of the Bride Part II (1995)	Comedy	1995

Parsing Timestamps

```
In [163... tags_timestamp = pd.read_csv('Datasets/tag.csv')
tags_timestamp.head()
```

```
Out[163...      userId  movied      tag      timestamp
```

0	18	4141	Mark Waters	2009-04-24 18:19:40
1	65	208	dark hero	2013-05-10 01:41:18
2	65	353	dark hero	2013-05-10 01:41:19
3	65	521	noir thriller	2013-05-10 01:39:43
4	65	592	dark hero	2013-05-10 01:41:18

```
In [165... tags_timestamp.dtypes
```

```
Out[165...  userId      int64
          movieId  int64
          tag      object
          timestamp object
          dtype: object
```

```
In [167... tags_timestamp['time'] = pd.to_datetime(tags_timestamp['timestamp'])
tags_timestamp.head()
```

```
Out[167...   userId  movieId      tag      timestamp      time
0      18     4141  Mark Waters  2009-04-24 18:19:40  2009-04-24 18:19:40
1      65      208   dark hero  2013-05-10 01:41:18  2013-05-10 01:41:18
2      65      353   dark hero  2013-05-10 01:41:19  2013-05-10 01:41:19
3      65      521  noir thriller  2013-05-10 01:39:43  2013-05-10 01:39:43
4      65      592   dark hero  2013-05-10 01:41:18  2013-05-10 01:41:18
```

```
In [168... tags_timestamp.dtypes
```

```
Out[168...  userId      int64
          movieId  int64
          tag      object
          timestamp object
          time      datetime64[ns]
          dtype: object
```

```
In [170... tags_timestamp_2015 = tags_timestamp[tags_timestamp['time'] > '2015-02-01']
tags_timestamp_2015.head()
```

```
Out[170...   userId  movieId      tag      timestamp      time
301     318      260   1970s  2015-02-20 22:42:49  2015-02-20 22:42:49
302     318      260  fantasy  2015-02-20 22:42:49  2015-02-20 22:42:49
303     318      260   sci-fi  2015-02-20 22:42:49  2015-02-20 22:42:49
304     318    115149   Action  2015-02-21 15:58:30  2015-02-21 15:58:30
305     318    115149  Revenge  2015-02-21 15:58:03  2015-02-21 15:58:03
```

```
In [171... tags_timestamp.shape, tags_timestamp_2015.shape
```

```
Out[171... ((465564, 5), (12130, 5))
```

```
In [174... tags_timestamp.sort_values(by = 'time', ascending=True)[:10]
```

Out [174...

	userId	movieId	tag	timestamp	time
333932	100371	2788	monty python	2005-12-24 13:00:10	2005-12-24 13:00:10
333927	100371	1732	coen brothers	2005-12-24 13:00:36	2005-12-24 13:00:36
333924	100371	1206	stanley kubrick	2005-12-24 13:00:48	2005-12-24 13:00:48
333923	100371	1193	jack nicholson	2005-12-24 13:02:51	2005-12-24 13:02:51
333939	100371	5004	peter sellers	2005-12-24 13:03:19	2005-12-24 13:03:19
333922	100371	47	morgan freeman	2005-12-24 13:03:32	2005-12-24 13:03:32
333921	100371	47	brad pitt	2005-12-24 13:03:32	2005-12-24 13:03:32
333936	100371	4011	brad pitt	2005-12-24 13:03:51	2005-12-24 13:03:51
333937	100371	4011	guy ritchie	2005-12-24 13:03:51	2005-12-24 13:03:51
333920	100371	32	bruce willis	2005-12-24 13:04:02	2005-12-24 13:04:02

Average Movie Ratings over Time

Movie ratings related to the year of launch?

In [175...

```
average_rating.head()
```

Out [175...

	movieId	rating
0	1	3.921240
1	2	3.211977
2	3	3.151040
3	4	2.861393
4	5	3.064592

In [176...

```
movies_data.head()
```


Out [176...

	movieId	title	genres	year
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1995
1	2	Jumanji (1995)	Adventure Children Fantasy	1995
2	3	Grumpier Old Men (1995)	Comedy Romance	1995
3	4	Waiting to Exhale (1995)	Comedy Drama Romance	1995
4	5	Father of the Bride Part II (1995)	Comedy	1995

In [179...

```
movies_data_final = movies_data.merge(average_rating, on = 'movieId' ,how =  
movies_data_final.head()
```

Out [179...

	movieId	title	genres	year	rating
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy	1995	3.921240
1	2	Jumanji (1995)	Adventure Children Fantasy	1995	3.211977
2	3	Grumpier Old Men (1995)	Comedy Romance	1995	3.151040
3	4	Waiting to Exhale (1995)	Comedy Drama Romance	1995	2.861393
4	5	Father of the Bride Part II (1995)	Comedy	1995	3.064592

In [181...

```
movies_data_final[['movieId','rating']].corr()
```

Out [181...

	movieId	rating
movieId	1.000000	-0.090369
rating	-0.090369	1.000000

In []: